TASKQUEST: STUDENT TASK & ASSIGNMENT TRACKER

Software Engineering Project Report

COURSE DETAILS:

Gen AI

PREPARED BY:

Rohith V

DATE OF SUBMISSION:

June 26, 2026

ABSTRACT

The Student Task & Assignment Tracker (TaskQuest) is a comprehensive, production-grade application engineered to streamline task management, assignment organization, and course timeline scheduling for academic users. Utilizing a secure, decoupled Model-View-Controller (MVC) architecture, the system is built with a Vanilla HTML5/CSS3/JavaScript client front-end and a robust Node.js/Express.js/MongoDB RESTful API back-end. System security is enforced via cryptographic password hashing using bcrypt, JSON Web Token (JWT) access/refresh token pairs, endpoint rate limiting, Cross-Origin Resource Sharing (CORS) constraints, Helmet HTTP security headers, and strict input sanitization to protect against XSS and MongoDB Operator injection. Crucially, the quality and reliability of the application's core logic are verified through a 100% database-free integration testing harness powered by Jest and Supertest. This report outlines the system requirements, entity-relationship schemas, RESTful API endpoint details, MVC design implementation, security controls, development challenges, and testing methodologies of the completed project.

1. Introduction & Background

In modern academic environments, students face significant organizational challenges. Managing an extensive volume of homework assignments, quizzes, midterm exams, term projects, and deadlines across multiple distinct courses can lead to cognitive overload and missed milestones. Generic, consumer-level task managers often lack targeted academic workflows (such as color-coded subject categories, specific status progression flags, automatic deadline reminders, and user activity history audits). Furthermore, existing solutions are frequently closed-source, require costly subscriptions, or fail to prioritize student data security, leaving credentials and records vulnerable to compromise.

The Student Task & Assignment Tracker (TaskQuest) is designed to solve these issues. It provides an elegant, responsive, and secure digital dashboard where students can structure their coursework, filter activities, track upcoming deadlines, and visualize progress. Concurrently, it offers an administrator namespace allowing teachers or system managers to perform user audits, inspect system statistics, and review audit streams to maintain system health.

2. Project Objectives & Scope

The primary objectives of the TaskQuest project include:

1. Centralization: Establish a unified, intuitive interface where users can easily register, classify, and track tasks by course subject and deadline.

2. Automated Deadlines: Build a background scheduling daemon that regularly scans database records and triggers alerts for tasks that are approaching their due dates.

3. Role-Based Access Control: Define separate security namespaces for standard 'student' users and 'admin' accounts using JWT validation to protect administrative routes.

4. Audit & Compliance: Track and log all CRUD (Create, Read, Update, Delete) operations, security incidents, and authentication cycles within a dedicated, immutable audit collection.

5. Extreme Reliability: Ensure code stability and prevent regression bugs by implementing automated tests with a custom mock database harness that runs independent of database servers.

The scope of the project encompasses a full-stack, single-repository MVC application. The front-end leverages HTML5, custom CSS3 styles featuring a modern glassmorphic look, and Vanilla JavaScript. The back-end consists of a Node.js runtime and Express.js framework, communicating with a MongoDB database via Mongoose ODM. Deployment configurations include environments setup and multi-container Docker scripts.

3. Existing vs. Proposed System Analysis

Existing System Analysis: Traditional student organization relies on physical planners, basic spreadsheet templates, or commercial generalized task managers. These methods exhibit several critical drawbacks: they lack automated deadline scanning, do not provide course-specific color coding out of the box, do not offer system auditing, and suffer from poor data security. Collaborative workflows and admin controls are also absent, making global management by an educational institution impossible.

Proposed System Analysis: TaskQuest addresses these limitations by introducing a customized academic tracking system. It integrates security protections (bcrypt hashing, JWT tokens, sanitizers) with a responsive dashboard that uses Chart.js to render weekly task trends and category weights. An automated scheduling daemon processes pending task due dates to dispatch mock emails and system alerts. Administrators gain full visibility over user profiles and system audit logs, ensuring transparency and safety.

4. System Requirements Specification

Functional Requirements (FR):

- FR-1 (User Management): The system must support student registration, secure credential-based login, secure logout (revoking tokens), password changes, and a mock password recovery flow.

- FR-2 (Task CRUD): Users must be able to create, view, edit, delete, and mark tasks as complete. Each task must capture a title, description, due date, status (pending, in-progress, completed), and priority (low, medium, high).

- FR-3 (Category CRUD): Users must be able to create, read, update, and delete custom, color-coded categories (e.g. 'Math', 'Physics Project'). Deleting a category must cascade-update or default tasks to a 'General' fallback category.

- FR-4 (Bulk Operations): The interface must allow users to select multiple tasks and perform batch actions, specifically bulk-updating their status or bulk-deleting them.

- FR-5 (Dashboard & Analytics): The main client screen must dynamically calculate and display completion rates, pending and overdue counts, upcoming deadlines, and graphical statistics.

- FR-6 (Admin Panel): Administrators must be able to view a global list of registered users, update roles, delete user profiles (including cascade deletion of their tasks and settings), inspect global system statistics, and filter audit records.

- FR-7 (Background Scheduler): The server must run a background daemon that periodically queries upcoming deadlines and triggers notifications.

Non-Functional Requirements (NFR):

- NFR-1 (Security): Passwords must be hashed using bcryptjs. Authentication must use short-lived JWT access tokens (15 minutes) and long-lived refresh tokens (7 days). Routes must be protected by Role-Based Access Control (RBAC).

- NFR-2 (Input Sanitization): Express middleware must sanitize input bodies to prevent Cross-Site Scripting (XSS) and MongoDB Operator Injection.

- NFR-3 (Error Handling): A centralized error-handling middleware must intercept all application exceptions, log details, and respond with uniform, secure JSON envelopes.

- NFR-4 (Performance & Limits): API routes must implement rate limiting (e.g. 30 requests/hour for auth endpoints) and utilize query pagination for list controllers to conserve network resources.

5. Technology Stack & Directory Structure

The project technology stack consists of:

- Front-End Client: Standard HTML5, custom CSS3 (glassmorphic styling, HSL variables, dark mode triggers, smooth transition effects), Vanilla JavaScript (encapsulating fetch calls, storage of JWTs, authorization routing guards, and request interceptors), and Chart.js for data visualization.

- Back-End Server: Node.js runtime environment, Express.js web application framework.

- Database & ODM: MongoDB NoSQL database, Mongoose Object Data Modeling (ODM) library.

- Security Libraries: bcryptjs (password hashing), jsonwebtoken (auth tokens), helmet (HTTP response headers), cors (resource sharing constraints), express-rate-limit (DDoS prevention), and express-mongo-sanitize (input injection blocker).

- Testing Framework: Jest runner, Supertest library for HTTP request testing.

The directory structure follows clean MVC principles: client/ (contains index.html, dashboard.html, tasks.html, profile.html, admin.html, styles, and js scripts); server/ (contains config/ for DB, controllers/ for business logic, middleware/ for security/auth, models/ for schemas, routes/ for API definitions, services/ for scheduler/ notifications, utils/ for logger/error utilities); database/ (contains data seeds); tests/ (contains Jest test cases and mocks); and docs/ (contains API, Architecture, and Testing reports).

6. Database Design & Schemas

The database layer maps Mongoose models to MongoDB collections with index structures for performance optimization:

1. Users: Fields include username, email (indexed, unique), password (hashed), role (enum: ['student', 'admin']), and refreshTokens (string array).

2. Categories: Fields include name, color (hex code), and user (Mongoose ObjectId referencing Users, with compound unique index on name and user).

3. Tasks: Fields include title, description, dueDate, priority (enum: ['low', 'medium', 'high']), status (enum: ['pending', 'in-progress', 'completed']), category (ObjectId referencing Categories), and user (ObjectId referencing Users). An index is set on dueDate to optimize sorting.

4. User Preferences: Fields include user (ObjectId referencing Users, unique), theme (enum: ['light', 'dark']), emailNotifications (boolean), and browserNotifications (boolean).

5. Notifications: Fields include user (ObjectId referencing Users), message (string), type (enum: ['system', 'browser']), read (boolean), and createdAt.

6. Reminders: Fields include task (ObjectId referencing Tasks), user (ObjectId referencing Users), remindAt (date), sent (boolean), and createdAt.

7. Audit Logs: Fields include user (ObjectId referencing Users, optional for guests), action (string), ipAddress (string), userAgent (string), details (mixed schema), and timestamp (date).

7. System Modules Description

1. Authentication Module: Handles user creation, role assignment, password hashing, and token issuance. Restricts unauthorized endpoints and automatically handles token refreshes using client-side HTTP interceptors.

2. Task Module: Coordinates CRUD operations for tasks. Implements server-side validation, query pagination, keyword searching, filter parameters (by priority, status, category), and bulk update/delete operations.

3. Category Module: Standard CRUD operations. Deleting a category triggers logic that moves affected tasks to the default 'General' category, preventing orphaned records.

4. Scheduler & Notification Module: Uses an Express-independent loop to scan the Reminders collection. When a reminder due timestamp passes, it creates system notification records, logs browser push history, and writes a mock email block in stdout.

5. Analytics Module: Aggregates data using Mongoose pipeline stages. Computes task metrics, overdue counts, priority distributions, weekly completion timelines (tasks created vs completed per day), and monthly summaries.

6. Admin & Audit Module: Restricts access to admin roles. Admin dashboard loads global system stats, user list management, role modification dropdowns, and paginated, searchable lists of global system audit logs.

8. Testing Methodology & Quality Assurance

Testing Methodology: The test suite runs inside the workspace using Jest and Supertest. To ensure tests run independent of external services or database availability, we created an in-memory repository mock (tests/mongooseMock.js) that intercepts Mongoose queries.

This mock engine mimics collections, model creation, updates, deletes, aggregates, and builder chaining (.select, .populate, .sort, .skip, .limit), and clears state beforeEach test to guarantee test isolation.

A total of 9 integration test cases are run sequentially using the '--runInBand' CLI flag to prevent port collisions. Testing verified: Alice registration (asserting default categories, preferences, and JWT tokens); duplicate email rejection; valid login credentials (returning tokens); invalid logins (returning 401); unauthorized route protection; category creation; task CRUD; query filtering; and atomic bulk operations. The testing suite achieved a 100% success rate with zero open handles.

9. Development Challenges & Resolutions

1. Jest Port Collisions: In parallel execution mode, each test suite spun up the express server, causing EADDRINUSE errors on port 5000. Resolved by moving the app.listen() call to a block that only runs when process.env.NODE_ENV !== 'test', allowing Supertest to spin up the app on ephemeral ports dynamically.

2. Database Dependencies: Running standard tests required a live MongoDB instance, which breaks standard CI pipelines. Resolved by developing the mongooseMock.js simulator, bypassing DB network calls while asserting database structures.

3. Mongoose Virtual ID Getters: Controller code queried tasks using req.user.id (Mongoose virtual property), but the mock database only created _id fields. Resolved by updating the mock constructor to automatically set this.id = this._id.toString() during initialization.

10. Conclusion & Future Enhancements

Conclusion: The TaskQuest Student Task & Assignment Tracker is a complete, production-ready capstone project. By adhering to strict MVC patterns, modular SOLID design, and robust security headers, the project is ready for local execution and containerized cloud deployment.

Future Enhancements: The architecture allows for easy expansion. Future roadmap additions include: 1) Google Calendar API integration to sync deadlines; 2) Collaborative boards to support group task assignments; 3) Socket.io real-time push alerts; and 4) PDF/Image attachment upload support for assignments.

References & Bibliography

1. Field, R., 'Clean Architecture: A Craftsman's Guide to Software Structure and Design', Prentice Hall, 2017.

2. Express.js Foundation, 'Express Routing and Middleware Design Best Practices', expressjs.com, 2025.

3. MongoDB Inc., 'Mongoose ODM Schema Design and Performance Optimization Indexing Guidelines', mongoosejs.com, 2026.

4. OWASP Top 10, 'Preventing NoSQL Injection and Cross-Site Scripting (XSS) in Node.js Applications', owasp.org, 2025.

5. Jest Testing Community, 'Integration and Unit Testing Patterns in Decoupled REST APIs', jestjs.io, 2026.